

# Selenium-based Browser Simulation for IDP Login – Analysis

## Selenium-based Browser Simulation for IDP Login – Analysis

### Goal:

Use **Selenium Grid** to simulate **real browser behavior** for login flows involving:

- JavaScript execution
- Dynamic redirects
- Token generation
- Cookie/session handling
- Web-auth flows like CAPTCHA, MFA, and CS Pages

### What Can Be Done with Selenium (Browser-Based Approach)

| Capability                | Status  | Notes                                                                                                          |
|---------------------------|---------|----------------------------------------------------------------------------------------------------------------|
| Full JavaScript Execution |         | Can handle all client-side logic, AJAX, dynamic token generation, and redirect behaviors.                      |
| Real Browser Sessions     |         | Mimics exact user behavior (Chrome/Firefox). Ensures accurate cookies, storage, session flows.                 |
| CAPTCHA (Visual)          | Partial | Can trigger CAPTCHA screens but cannot auto-solve without third-party services or test bypass flags.           |
| MFA Handling              | Partial | Can simulate OTP entry <i>if</i> test OTP is retrievable via backend or mock. Otherwise requires manual input. |
| UI Flow Automation        |         | Ideal for full web registration, contribution, withdrawal flows where form events, DOM updates occur via JS.   |
| Session/Token Chaining    |         | Natural in Selenium — browser handles it natively like a user.                                                 |
| Screenshots & Debug Logs  |         | Useful for debugging and demonstrating test coverage.                                                          |
| Integration with Taurus   |         | Can run via Taurus YAML config and embed in CI/CD to some extent.                                              |

### Limitations & Challenges of Selenium for Load Simulation

| Limitation                             | Impact                                                                                                             |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Not Designed for Load Testing          | Cannot simulate hundreds/thousands of users like JMeter. Not scalable for stress/load testing.                     |
| Grid Capacity Limited                  | You mentioned <b>2 nodes × 30 max = 60 users</b> – suitable for functional validation, not full concurrency tests. |
| High Infrastructure Cost               | Requires VMs/containers for each browser session resource-heavy and slow execution.                                |
| Test Suite Needs to Be Developed       | No suite exists today — scripting + validations + synchronization logic will take time to build.                   |
| Maintenance Heavy                      | UI changes, element IDs, workflows require frequent script updates — brittle in fast-moving apps.                  |
| Poor Reporting for Performance Metrics | No built-in SLA, latency %, or concurrency graphs like JMeter/NeoLoad — needs additional work to collect /report.  |

### Summary Comparison: Selenium vs JMeter for Current Need

| Feature                             | JMeter            | Selenium          |
|-------------------------------------|-------------------|-------------------|
| Load/Stress Testing                 | Designed for this | Not suitable      |
| JS-based Token Handling             | Not Supported     | Fully Supported   |
| Session/Redirect Mimic              | Manual Setup      | Natural           |
| CAPTCHA/MFA                         | Unsupported       | Semi-supported    |
| Complex UI Flow (Web Reg, Withdraw) | Hard              | Ideal             |
| CI/CD Ready                         | Strong            | Needs Infra Setup |
| Reporting (Latency, TPS)            | Inbuilt           | Manual            |

|                |             |                     |
|----------------|-------------|---------------------|
| Infrastructure | Lightweight | Heavy (per browser) |
|----------------|-------------|---------------------|

## Recommendations

- Use **JMeter + existing working suite** for **IDP load tests**, where JavaScript is not a hard dependency.
- Consider **Selenium Grid** for **complex flows like web registration** or token-driven UIs where browser mimic is essential.
- Combine both: Let **Selenium handle session creation** (via browser), and pass tokens into **JMeter load scripts** to simulate scalable login-based flows.
- Plan and build Selenium test suites only for scenarios that **cannot be replicated with HTTP clients alone**.